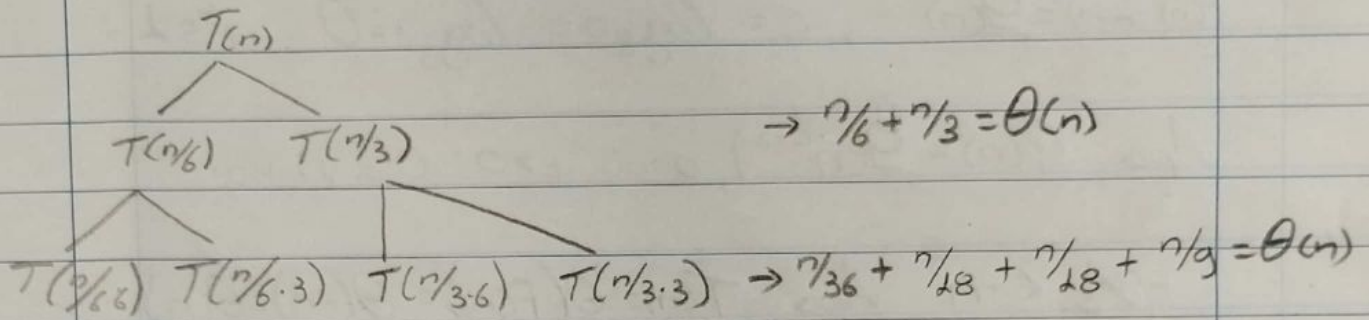


Αλγόριθμοι και Πολυπλοκότητα 2^ο φύλλο 5.10

1. $T(n) = T(n/6) + T(n/3) + n$, $(T(1) = \Theta(1))$



Υψος δέντρου $n/3^h = 1 \Rightarrow n = 3^h \Rightarrow h = \log_3 n$

Αρα έχουμε ότι το δέντρο έχει υψος $\log_3 n \rightarrow \Theta(\log_3 n)$
και αφού κάθε επίπεδο έχει $\Theta(n)$, ο συνολικός χρόνος είναι $\Theta(n \log_3 n)$

2. (i) $T(n) = 4T(n/2) + n^2 + \log n$

$a=4, b=2, f(n) = n^2 + \log n$
 $c = \log_b a = \log_2 4 = 2 \log_2 2 = 2$ (i)

$f(n) = \Theta(n^2)$ (ii) Αρα συγκρίνουμε n^c και $\Theta(n^2)$
 $n^2 = \Theta(n^2)$

Αρα $T(n) = \Theta(n^c \log n) = \Theta(n^2 \log n)$

(ii) $T(n) = \log(n)T(n/2) + n$. Εδώ το a δεν είναι σταθερός όρος άρα δεν μπορεί να εφαρμοστεί το Master Theorem

$$(iii) \quad T(n) = T(n/3) - n$$

$$a=1, b=3, F(n) = (-n)$$

$$F(n) \neq n = O(n), \quad c = \log_3 a = \log_3 1 = 0, \quad n^c = 1$$

$$\text{Αρα } F(n) = O(n^{c+\epsilon}), \text{ οπου } \epsilon > 0 \text{ (π.χ. } \frac{1}{2}) \text{ και}$$

$$-n/3 < -n \quad \text{Αρα } T(n) = \Theta(F(n)) = \Theta(-n) = \Theta(n)$$

$$(iv) \quad T(n) = 3T(n/3) + \sqrt{n} + \log n$$

$$a=3, b=3, F(n) = \sqrt{n} + \log n$$

$$c = \log_3 3 = 1, \quad F(n) = O(\sqrt{n})$$

$$\text{Υποκει } \epsilon > 0 \text{ (π.χ. } \epsilon = 1/2) \text{ τότε } n^{c-\epsilon} = n^{1-\epsilon} = O(\sqrt{n})$$

$$\text{Αρα } T(n) = \Theta(n^c) = \Theta(n)$$

$$(v) \quad T(n) = T(4n/5) + n \log n + \sqrt{n}$$

$$a=4, b=5/4, F(n) = n \log n + \sqrt{n}$$

$$c = \log_{5/4} 4 = 0, \quad n^c = 1, \quad F(n) = O(n \log n)$$

$$F(n) = O(n^{0+\epsilon}), \epsilon > 0 \text{ και } a F(n/b) = \frac{5n}{4} \log(5/4) < n \log(n)$$

(6)

(vii) $T(n) = 2T(n/3) + 3T(2n/3) + n$. Δεν μπορεί να εφαρμοστεί το Masters Theorem διότι δεν είναι της μορφής $T(n) = aT(n/b) + f(n)$

3. Αλγόριθμος FindPivotElement

Είσοδος: Πίνακας A

Έξοδος: Θύρα i ενός στοιχείου

left = 0

right = length(A) - 1

while left <= right

mid = (left + right) DIV 2

if (mid == 0 || A[mid] >= A[mid-1]) && (mid == length(A) || A[mid] >= A[mid+1])
return mid

if (mid > 0) && A[mid-1] > A[mid]

right = mid - 1

else

left = mid + 1

return -1

4. Αλγόριθμος MinElementsGreaterSum

Είσοδος: Πίνακας A kn αρνητικών

Έξοδος: Πλήθος στοιχείων που έχουν sum > sum(A)

A.sort(reverse=True) # φθίνουσα ταξινότηση $O(n \log n)$

total_sum = sum(A)

current_sum = 0

count = 0

For num in A // $O(n)$

current_sum += num

count += 1

if current_sum > total_sum - current_sum:

return count

// $O(n \log n) + O(n) = O(n \log n)$

5. Αλγόριθμος MST With Edge

Είσοδος: Γράφος $G(V, E)$, ακμή e

Εξοδος: Ελάχιστο συνδεδεμένο δέντρο T που περιέχει την e

1. $T \leftarrow \{e\}$ $\rightarrow$ disjoint_set

2. Δημιουργία συνόλου διαμερισμών S , με κάθε κορυφή $v \in V$ ξεχωριστό σύνολο

3. Ένωση των συνόλων που περιέχουν τις κορυφές της e στο S

4. $E' \leftarrow$ ταξινομημένη λίστα των ακμών του $E - \{e\}$ κατ'αύξουσα σειρά βάρους

5. For edge in edges

6. if edge != required_edge

7. $u, v, \text{weight} = \text{edge}$

8. if disjoint_set.find(u) != disjoint_set.find(v)

9. mst.append(edge)

10. disjoint_set.union(u, v)

11. return mst

6. Algorithm count_paths(A):

Είσοδος: Πίνακας A

Έξοδος: Πλήθος μονοπατιών

$n = \text{length}(A)$

$dp = [[0 \text{ for } _ \text{ in range}(n)] \text{ for } _ \text{ in range}(n)]$

2D array, $n \times n$, αρχικοποίηση με 0

if $A[0][0] == 0$

Αρχικοποίηση του κελιού

$dp[0][0] = 1$

else

return 0 # no paths

for j in range(1, n) # Γεμίζω πρώτα γραμμή

if $A[0][j] == 0$

$dp[0][j] = dp[0][j-1]$

else

break

Αν τυχούσε 1

for i in range(1, n) # Γεμίζω πρώτα στήλη

if $A[i][0] == 0$

$dp[i][0] = dp[i-1][0]$

else

break

Αν τυχούσε 1

for i in range(1, n) # Γεμίζω τον υπολοιπό πίνακα

for j in range(1, n)

if $A[i][j] == 0$

$dp[i][j] = dp[i-1][j] + dp[i][j-1]$

return $dp[n-1][n-1]$